Week 13 Lab Handout

Versioning — Shipping a Breaking Change Without Breaking Everyone

Time: 110 minutes

Bring: your laptop, your API from Weeks 5–12 (auth, rate limiting, logging, `/stats`), and your ring partner. Today you are both the one who ships a breaking change and the one who receives it

---

Before you start: today is deliberately uncomfortable

> For twelve weeks you have built an API that your partner depends on, and depended on theirs. Today each of you will change yours in a way that breaks the other person's code, and each of you will get a change you did not ask for. That is not a mistake in the schedule. It is the most realistic thing in the course.

The skill is not avoiding breaking changes, because every API that lives long enough needs one. The skill is **shipping them so nobody is hurt**: the new version runs *beside* the old one, the old one keeps working, the change is announced clearly, and consumers get time to move.

Two things happen in parallel today. You will **send** a notice to your downstream partner (Parts A to D), and you will **receive** one from your upstream partner and migrate (Part E). Send yours early. The sooner it arrives, the more time your partner has.

Part A — Choose One Breaking Change (10 min)

First, the line between safe and breaking. A change is **breaking** if code that worked yesterday could fail today, without the caller changing anything.

Change	Breaking?	Why
Add a new optional field to a response	No	Callers ignore fields they don't know
Add a new endpoint, or a new optional query parameter	No	Nothing existing is touched
Rename a field (`dueDate` to `due`)	**Yes**	Code reading `dueDate` now gets `undefined`
Remove a field	**Yes**	Same, and worse
Change a type (a string date becomes a number)	**Yes**	Parsing and comparisons break
Wrap a bare list in an object (`[...]` becomes `{data: [...]}`)	**Yes**	Code that loops over the list now fails
Make an optional request field required	**Yes**	Old requests start getting 400
Change what a status code means (200 becomes 201)	**Yes**	Code that checks `=== 200` now fails

Now pick **one** improvement to your own API that needs a breaking change. It must be *legitimate*: a real reason, not change for its own sake. Good reasons sound like these:

- Wrap lists in `{ data, count }`, so you can add paging later without another break
- Fix an inconsistent name (`dueDate` in one place, `due_date` in another)

- Return real timestamps instead of loose date strings
- Split one overloaded endpoint into two clear ones

Keep it **small**. One change, not a redesign. A small change keeps the migration small, and your partner has about thirty minutes.

Create `VERSIONING_NOTES.md` in your repo root and write:

1. **The change,** in one sentence.
2. **Why it is worth a break.**
3. **A prediction:** which exact line in your partner's code will fail? Look at how their app uses your API (their Week 10 integration). If you can't name the line, ask them. This is the first time you have to think about your API *from their seat*.

This handout's examples use the Task Tracker, with two changes: the list is wrapped in `{ data, count }`, and `dueDate` becomes `due`. Yours can differ.

Part B — Build v2 Beside v1 (30 min)

Step 1: take a snapshot of v1, before you touch anything. This is your proof later that v1 did not change:

```
curl -s -H "X-API-Key: <your-real-key>" \
  http://localhost:3000/tasks > v1_before.json
```

Step 2: move your existing handlers into `v1.js`. Create a router, and paste your current `/tasks` handlers into it **exactly as they are**. Move, don't edit. Every edit to v1 is a chance to break the people still using it.

```
// v1.js: FROZEN. Do not change the shape of anything in this file.
const express = require('express');
const router = express.Router();

// ...your existing GET /tasks and GET /tasks/:id handlers, pasted unchanged

module.exports = router;
```

Use `router.get('/tasks', ...)` where your app had `app.get('/tasks', ...)`. If your handlers read from a shared data file or database module, keep requiring it. Both versions should read the *same* data.

Step 3: write `v2.js`. It is a different *view* of the same data, not a copy of your whole app. One small function turns a stored task into the new shape, so the change lives in exactly one place:

```
// v2: the new shape. Same data, different response.
const express = require('express');
const { tasks } = require('./store');

const router = express.Router();

// One place that turns a stored task into the v2 response shape.
const toV2 = t => ({
  id: t.id,
  title: t.title,
  due: t.dueDate,          // renamed: was "dueDate" in v1
  status: t.status,
});

router.get('/tasks', (req, res) => {
  const { status } = req.query;
  const rows = status ? tasks.filter(t => t.status === status) : tasks;
  res.json({ data: rows.map(toV2), count: rows.length });   // was a bare list
});

router.get('/tasks/:id', (req, res) => {
```

```
  const task = tasks.find(t => t.id === Number(req.params.id));
  if (!task) return res.status(404).json({ message: 'Task not found' });
  res.json({ data: toV2(task) });
});

module.exports = router;
```

Step 4: mount both, after your key check. Order matters again. If you mount `v2` before `requireApiKey`, the new version answers anyone, with no key:

```
const v1 = require('./v1');
const v2 = require('./v2');

app.use(monitor);
app.use(requireApiKey);
app.use(rateLimit({ windowMs: 60000, max: 100 }));
app.get('/stats', statsHandler);

app.use('/v2', v2);      // the new version
app.use('/v1', v1);      // the old version, still running
app.use('/', v1);        // old URLs with no version (/tasks) keep working
```

The last line is the one people forget. Your partner's code calls `/tasks` *today*. If you only mount v1 at `/v1`, `/tasks` stops working the moment you deploy, and you have broken them with no warning. That is exactly the thing versioning exists to prevent.

Step 5: make your log show the version. A router mounted at `/v2` removes that prefix from `req.path`, so your Week 12 log would print `/tasks` for both versions. In `monitoring.js`, change the one line:

```
      path: req.baseUrl + req.path,    // still no query string
```

Your `/stats` routes already include the prefix, so `GET /v1/tasks`, `GET /tasks` and `GET /v2/tasks` appear as three rows. You'll use that in Part F.

Step 6: try all three. Restart and call each URL:

```
curl -s -H "X-API-Key: <your-real-key>" \
  "http://localhost:3000/v1/tasks?status=active"
curl -s -H "X-API-Key: <your-real-key>" \
  "http://localhost:3000/v2/tasks?status=active"
curl -s -H "X-API-Key: <your-real-key>" \
  "http://localhost:3000/tasks?status=active"
```

The first and third must return the same old shape. The second returns the new one:

```
{"data":[{"id":1,"title":"Write report","due":"2026-11-02","status":"active"},
{"id":2,"title":"Book lab room","due":"2026-11-05","status":"active"}],
"count":2}
```

Part C — Prove v1 Didn't Change, and Say It's Leaving (15 min)

"I think v1 is unchanged" is not evidence. Get two kinds.

Evidence 1: a diff. Fetch v1 again and compare it to the snapshot from Part B:

```
curl -s -H "X-API-Key: <your-real-key>" \
  http://localhost:3000/tasks > v1_after.json
diff v1_before.json v1_after.json && echo "v1 unchanged"
```

If `diff` prints anything, you changed v1. Fix it before going on.

Evidence 2: a test that stays. The diff proves it once. A test proves it every time. Save this as `versions.test.js` (it uses the same Jest and supertest setup as your earlier tests, and needs your `app` exported from `app.js` without calling `listen`):

```
process.env.API_KEYS = 'test-key';
const request = require('supertest');
const app = require('./app');

const KEY = { 'X-API-Key': 'test-key' };

describe('v1 is frozen', () => {
  test('GET /v1/tasks keeps its original shape', async () => {
    const res = await request(app).get('/v1/tasks').set(KEY);
    expect(res.status).toBe(200);
    expect(Array.isArray(res.body)).toBe(true);
    expect(Object.keys(res.body[0]).sort())
      .toEqual(['dueDate', 'id', 'status', 'title']);
  });

  test('old unversioned /tasks still answers the same way', async () => {
    const a = await request(app).get('/tasks').set(KEY);
    const b = await request(app).get('/v1/tasks').set(KEY);
    expect(a.status).toBe(200);
    expect(a.body).toEqual(b.body);
  });

  test('v1 error shape is unchanged', async () => {
    const res = await request(app).get('/v1/tasks/999').set(KEY);
    expect(res.status).toBe(404);
    expect(res.body).toEqual({ message: 'Task not found' });
  });
});

describe('v2 has the new shape', () => {
  test('list is wrapped, with "due" instead of "dueDate"', async () => {
    const res = await request(app).get('/v2/tasks').set(KEY);
    expect(res.status).toBe(200);
    expect(res.body.count).toBe(3);
    expect(Object.keys(res.body.data[0]).sort())
      .toEqual(['due', 'id', 'status', 'title']);
  });

  test('status filter still works', async () => {
    const res = await request(app).get('/v2/tasks?status=active').set(KEY);
    expect(res.body.data.map(t => t.id)).toEqual([1, 2]);
  });
});

describe('deprecation headers', () => {
  test('v1 and the old URLs announce the sunset', async () => {
    for (const path of ['/v1/tasks', '/tasks', '/v1/tasks/1']) {
      const res = await request(app).get(path).set(KEY);
      expect(res.headers.deprecation).toBe('true');
      expect(res.headers.sunset).toBe('Fri, 13 Nov 2026 00:00:00 GMT');
      expect(res.headers.link).toContain('rel="successor-version"');
    }
  });

  test('v2 and unknown paths do not carry them', async () => {
    for (const path of ['/v2/tasks', '/nope']) {
      const res = await request(app).get(path).set(KEY);
      expect(res.headers.deprecation).toBeUndefined();
    }
  });
});
```

Run `npx jest`. Notice what the first group pins down: the **exact set of field names** in v1. If anyone ever "tidies" v1 by accident, this fails.

Now add the announcement to v1 itself. Real APIs don't only send an email. They tell every caller, on every response, that the endpoint is going away. Create `deprecate.js`:

```
// Adds the standard "this is going away" headers to a response.
function deprecated({ sunset, successor }) {
  return (req, res, next) => {
    res.set('Deprecation', 'true');
    res.set('Sunset', new Date(sunset).toUTCString());
    res.set('Link', `<${successor}>; rel="successor-version"`);
    next();
  };
}

module.exports = { deprecated };
```

Then use it in `v1.js`. Add the import and the `old` line near the top, and put `old` as the second argument of each route. The handlers themselves stay as they are:

```
// v1: FROZEN. Do not change the shape of anything in this file.
const express = require('express');
const { tasks } = require('./store');
const { deprecated } = require('./deprecate');

const router = express.Router();
const old = deprecated({ sunset: '2026-11-13', successor: '/v2/tasks' });

router.get('/tasks', old, (req, res) => {
  const { status } = req.query;
  res.json(status ? tasks.filter(t => t.status === status) : tasks);
});

router.get('/tasks/:id', old, (req, res) => {
  const task = tasks.find(t => t.id === Number(req.params.id));
  if (!task) return res.status(404).json({ message: 'Task not found' });
  res.json(task);
});

module.exports = router;
```

Three headers now ride along on every v1 response: `Deprecation` says "this is deprecated", `Sunset` says the date it will stop working, and `Link` points to the replacement. Check them:

```
curl -si -H "X-API-Key: <your-real-key>" http://localhost:3000/tasks | head -6
```

```
HTTP/1.1 200 OK
X-Powered-By: Express
Deprecation: true
Sunset: Fri, 13 Nov 2026 00:00:00 GMT
Link: </v2/tasks>; rel="successor-version"
```

Use *your* sunset date, which you choose in Part D. Run `npx jest` again. Everything should pass, and a request to `/v2/tasks` must **not** carry these headers.

Part D — Write and Send Your Deprecation Notice (15 min)

Your partner will read this in a hurry, maybe on their phone, with their own work to do. Here is the **bad** version, which is how most breaking changes actually arrive:

> hey we changed the tasks endpoint, pls update ■

It doesn't say what changed, why, how to fix it, or by when. Your partner has to find out by breaking, then guess. Now the **good** version. Create `DEPRECATION_NOTICE.md` with these sections, filled in for your API:

```
# Deprecation notice: Tasks API v1
```

```
To: <partner name>        From: <your name>        Date: <today>

## Summary
v1 of GET /tasks is deprecated. v2 is live now. v1 will stop working on
<sunset date>. Please move before then.

## What is changing
- The list is wrapped: [ ... ] becomes { "data": [ ... ], "count": N }
- The field "dueDate" is renamed "due"
- Nothing else changes: ids, titles, status values, your API key

## Why
<One honest sentence. Example: wrapping the list lets us add paging later
without another breaking change.>

## Before and after
v1  GET /tasks
    [ { "id": 1, "title": "Write report", "dueDate": "2026-11-02" } ]

v2  GET /v2/tasks
    { "data": [ { "id": 1, "title": "Write report", "due": "2026-11-02" } ],
      "count": 1 }

## How to migrate
1. Call /v2/tasks instead of /tasks. Same key, same ?status= filter.
2. Read the list from body.data instead of from the body itself.
3. Read "due" where you read "dueDate".
4. Test it, then tell me, so I can see your calls in my logs.

## Timeline
- Today: v2 is live, v1 still works and now sends Deprecation headers
- <sunset date>: v1 stops working

## Questions
Message me any time. I will reply within <time>.
```

Four things make this a good notice: it says **what** changed in terms your partner can search their own code for, **why**, an **example of the new shape** they can copy, and a **date**. It also says what is *not* changing, which saves them from checking everything.

Choosing the sunset date. Too short and your partner can't finish. Too long and you carry two versions forever. Real companies give months. In this course, pick a date your partner can honestly meet, with a buffer, and **no later than the day before Demo Day**, so that both rings present on v2. In this course v1 stays running until after Demo Day so nobody's demo breaks. Write the notice as if you meant the date, because in real life you would.

Send it properly:

1. Commit `DEPRECATION_NOTICE.md` to your repo so there is a permanent copy.
2. Send your partner the **full text**, plus a link to the file. Don't send only a link, and don't send only "see the repo".
3. Ask them to reply when they've read it. A notice that was never received hasn't been sent.

Part E — Receive a Notice and Migrate (30 min)

Now the other side. Your upstream partner has changed their API and sent you a notice. Your Week 10 and 11 feature, the one that calls their API, is about to break unless you act.

Step 1: read the whole notice, then judge it. Does it say what changes, why, an example, and a date? Write one or two sentences in `VERSIONING_NOTES.md` about how it felt to receive it. You'll need this for Part F. If something is missing, ask **one specific question** (for example "does `?status=` still work on v2?"), not "can you explain?". Note the sunset date.

Step 2: try the new version by hand, before changing any code:

```
curl -s -H "X-API-Key: <partner-key>" "<partner-url>/v2/tasks?status=active"
```

Compare the output to the example in their notice. If it doesn't match, tell them now.

Step 3: put the partner's shape in one place. Right now the shape of their response is probably spread through your route. Create `partner.js`, so the *only* file that knows about v1 or v2 is this one. Adjust the paths and field names to your partner's notice:

```
// The ONE place that knows the shape of your partner's API.
const versions = {
  v1: {
    path: '/tasks',
    parse: body => body.map(t => ({ title: t.title, dueDate: t.dueDate })),
  },
  v2: {
    path: '/v2/tasks',
    parse: body => {
      if (!Array.isArray(body.data)) throw new Error('unexpected v2 shape');
      return body.data.map(t => ({ title: t.title, dueDate: t.due }));
    },
  },
};

module.exports = { versions };
```

Two details matter. Each version has its own `parse`, so the rest of your app always receives the same thing: `{ title, dueDate }`. And the v2 parser **checks the shape** and says so if it's wrong, instead of failing with a vague error somewhere else.

Step 4: use it in your `/upcoming` route. Keep the Week 11 code. Only the URL and the parsing change. Pick the version from `.env`:

```
const { versions } = require('./partner');
const ver = versions[process.env.PARTNER_VERSION || 'v1'];

// inside the route, replacing the old URL and the old .map(...):
const r = await fetchWithRetry(
  `${PARTNER_URL}${ver.path}?status=active`,
  { headers: { 'X-API-Key': PARTNER_KEY } },
  { timeoutMs: 1500, retries: 2, delayMs: 200 }
);
if (!r.ok) throw new Error(`partner returned ${r.status}`);
const data = ver.parse(await r.json());
```

Add `PARTNER_VERSION=v1` to `.env.example` (never a real key). Now the migration is a **one-word change** and so is the rollback.

Step 5: test the parsers. Save this as `partner.test.js` and run `npx jest`:

```
const { versions } = require('./partner');

const v1Body = [{ id: 1, title: 'Write report', dueDate: '2026-11-02' }];
const v2Body = {
  data: [{ id: 1, title: 'Write report', due: '2026-11-02' }],
  count: 1,
};
const wanted = [{ title: 'Write report', dueDate: '2026-11-02' }];

test('v1 and v2 give the page exactly the same thing', () => {
  expect(versions.v1.parse(v1Body)).toEqual(wanted);
  expect(versions.v2.parse(v2Body)).toEqual(wanted);
});

test('the wrong parser fails loudly, not quietly', () => {
  expect(() => versions.v1.parse(v2Body)).toThrow();
```

```
  expect(() => versions.v2.parse(v1Body)).toThrow('unexpected v2 shape');
});
```

Step 6: switch, and watch for the quiet failure. Set `PARTNER_VERSION=v2` in `.env`, restart, and load your page. Here is the trap: your Week 11 fallback is so good that a *broken* migration can hide. If `parse` throws, the route catches it and shows **saved data from before**. The page looks fine, with a banner you might not read. So check, on purpose:

- The response has `"stale": false`, not `true`
- Your terminal shows **no** `partner call failed` line
- Switch back to `PARTNER_VERSION=v1`, reload, and switch to `v2` again. Both should work until the sunset date

Step 7: show your partner it landed. Ask them to read their `/stats` or their log. They should see a row `GET /v2/tasks` with *your* `caller` label, and the count on `GET /tasks` should stop growing. That is the real evidence that you migrated, and it's what they need before they can ever turn v1 off. Take a screenshot or copy the lines.

Step 8: log it. Add a row to your `INTEGRATION_LOG.md`: the symptom (a deprecation notice arrived), what changed, what you changed, and whose code it was. Point out one thing: this time nothing *broke*, because you were warned.

Part F — Reflect and Push (10 min)

Who still uses v1? Your own `/stats` can answer a question real teams ask before they turn something off. Read it:

```
curl -s -H "X-API-Key: <your-real-key>" http://localhost:3000/stats
```

Look at the rows `GET /v1/tasks` and `GET /tasks`, and at your log lines for those paths. Write down: how many calls, and from which `caller` labels. Is your partner still on v1? Could you switch v1 off today without breaking anyone?

The reflection. Add a section to `VERSIONING_NOTES.md`, written in your own words, about 150 to 250 words:

1. What was it like to receive a breaking change on short notice? How did you find out, and what did you do first?
2. What did your partner's notice do well, and what was missing?
3. Read your own notice again **as your partner**. What would you change?
4. As the API owner next time, what is one thing you would do differently?

Specific beats generic. "It was annoying" doesn't help anyone, and "Mine was fine" isn't perspective-taking. "I found out when my page showed saved data, and I didn't know whether it was my bug or theirs" is.

Check and push:

```
git status
```

`.env` must **not** be listed. Then:

```
git add . v1.js v2.js deprecate.js partner.js versions.test.js partner.test.js
git add VERSIONING_NOTES.md DEPRECATION_NOTICE.md INTEGRATION_LOG.md
git commit -m "Week 13: v2 beside v1, deprecation notice, migrate to partner v2"
git push
```

This week's deliverable checklist

- `/v2/` works with the new shape, and `/v1/` (plus the old `/tasks` URL) still returns the old shape
- `diff v1_before.json v1_after.json` printed nothing, and `versions.test.js` passes
- v1 responses carry `Deprecation`, `Sunset` and `Link` headers, and v2 does not
- `/v2/` needs an API key like everything else
- `DEPRECATION_NOTICE.md` covers what, why, a before-and-after example, migration steps, and a sunset date, and was sent to your partner
- Your integration calls your partner's v2, with `partner.js` and a passing `partner.test.js`
- Evidence of the migration: `"stale": false`, plus your partner's `/stats` or log showing your calls on v2
- `VERSIONING_NOTES.md`: the change, your prediction, who still uses v1, and the reflection
- `INTEGRATION_LOG.md` updated, and everything committed and pushed

Rubric (25 pts)

Criterion	Points
v2 implemented correctly, v1 still functional (non-breaking rollout)	8
Deprecation notice is clear, complete, professional	6
Successfully migrated to upstream partner's v2	8
Reflection shows genuine perspective-taking	3

Common problems and quick fixes

Problem	Likely cause and fix
Your partner's `/tasks` calls started failing	You mounted v1 only at `/v1`. Add `app.use('/', v1)`
`Cannot GET /v2/tasks`	`v2.js` doesn't end with `module.exports = router`, or `app.use('/v2', v2)` is missing
`/v2/tasks` answers with no key	`v2` is mounted above `requireApiKey`. Move it below
`diff` shows v1 changed	You edited a handler while moving it. Run `git diff` and put it back
No `Deprecation` header on v1	`old` isn't the second argument of the route, or you called `/v2`
After switching to v2 the page says "saved data"	`parse` threw. Read the `partner call failed` line, then compare their real body to your parser
Your partner's notice never arrived	Message them with a specific ask. If it's still silent, tell your instructor

Next week: ***Demo Day.*** *Your API and your partner's, live, in ring order. Before then, make sure* `run.sh` *starts cleanly from a fresh clone, and that your integration points at your partner's v2.*